AR HUD System

- Author : Sunitha Kosireddy
- AAI 590 Capstone project, April 2026

omniview_e2e_live_v3.ipynb — AR HUD System

This is the end-to-end runner for the OmniView AR HUD system. It orchestrates the full pipeline from setup to output across two operating modes. The notebook contains 22 cells organized into 6 - **sequential stages:**

- **Setup Cell** — Clones the project repo from GitHub, installs dependencies (YOLOv8, OpenCV, ImageIO/FFmpeg), and verifies the source video file is present.
- **Cell #2 (Patch Cell)** — Applies 5 instrumentation patches to omniview_e2e_live.py, injecting performance counters for detections, blind spot alerts, fast-approach events, and per-frame render timing. Saves results to a JSON file after the pipeline runs.
- **Mode 1 Cell (JSON File Mode)** — Runs the HUD pipeline in offline mode, reading pre-recorded detection data from JSON files and rendering the AR overlay onto the video.
- **DeepStream Cell** — Launches the live DeepStream dual-model pipeline (LISA + COCO) as a background process, patching it for the Colab environment and waiting for it to start before proceeding.
- **Mode 2 Cell (UDP Live Mode)** — Runs the HUD pipeline in live mode, receiving real-time detections from the DeepStream pipeline over UDP port 5055.
- **Report Cell + GIF Cell** — Reads the saved JSON performance files from both modes and renders a formatted side-by-side comparison report. Also displays the animated GIF outputs from both modes inline in the notebook.

This code clones the project repo from GitHub if it isn't already present locally. Second, it installs the required dependencies — Ultralytics (YOLOv8), OpenCV, and ImageIO with FFmpeg support for video processing. Third, it locates the output video file output_ds_1.mp4 inside the repo and checks that it's larger than 1 MB. If the file is missing or too small, it prompts the user to manually upload it via Colab's file uploader and moves it into the repo directory. Finally, it confirms setup is complete.

- location for output_ds_1.mp4 and output_ds_2.mp4 or other mp4 files when prompted : upload this file from here into run area

https://drive.google.com/drive/folders/1i3iuQBTVqTwLLOUI8QtTs_MY4I1Zkign?dmr=1&ec=wgc-drive-%5Bmodule%5D-goto

Instructions to run the code: For File Mode 1: Json File mode

1. Run Setup cell
2. Run cell #2
3. Run: Mode 1 Cell (Json Mode)
4. Run: Report Cell

For File Mode 2 : UDP mode

1. Run : Setup cell
2. Run : cell #2 (Don't run cell #3)
3. Run: Deep Stream Cell
4. Run : Mode 2 Cell (UDP mode)
5. Run : Report Cell

- Finally Run: This will run gif generate dfrom Mode1 and Mode 2
  - Run : Gif run cell

Set Up Cell

```
import os
import shutil
from pathlib import Path

# ============================================================
# SETUP
# ============================================================
REPO = "/content/AAI-590-Capstone-group2"
REPO_URL = "https://github.com/TEJSINGH17/AAI-590-Capstone-group2.gi
DEFAULT_VIDEO_NAME = "output_ds_1.mp4"

# ============================================================
# 1. CLONE REPO IF NEEDED
# ============================================================
if not os.path.exists(REPO):
    os.system(f"git clone {REPO_URL} {REPO}")
    print("Repo cloned")
else:
    print("Repo already exists")

# ============================================================
# 2. INSTALL DEPENDENCIES
# ============================================================
```

```
os.system("pip install ultralytics opencv-python-headless imageio im
print("Dependencies ready")

# ============================================================
# 3. FIND OR UPLOAD VIDEO
# ============================================================
default_video_path = os.path.join(REPO, DEFAULT_VIDEO_NAME)
video_path = None

# Option A: use default repo video if available
if os.path.exists(default_video_path) and os.path.getsize(default_vi
    video_path = default_video_path
    print(f"Using default repo video: {video_path}")
    print(f"Video size: {os.path.getsize(video_path)/1e6:.1f} MB")

# Option B: upload any mp4 file
else:
    print("Default video not found. Please upload an MP4 file.")
    from google.colab import files

    uploaded = files.upload()

    if not uploaded:
        raise FileNotFoundError("No file was uploaded.")

    uploaded_mp4 = None
    for fname in uploaded.keys():
        if fname.lower().endswith(".mp4"):
            uploaded_mp4 = fname
            break

    if uploaded_mp4 is None:
        raise ValueError("Uploaded file is not an MP4. Please upload

    src = uploaded_mp4
    dst = os.path.join(REPO, uploaded_mp4)

    # Remove destination first if it already exists
    if os.path.exists(dst):
        os.remove(dst)

    shutil.move(src, dst)
    video_path = dst

    print(f"Using uploaded video: {video_path}")
    print(f"Video size: {os.path.getsize(video_path)/1e6:.1f} MB")

# ============================================================
# 4. MAKE A STANDARD NAME COPY
#    This helps if later code expects output_ds_1.mp4
# ============================================================
```

```
standard_video_path = os.path.join(REPO, DEFAULT_VIDEO_NAME)

if video_path != standard_video_path:
    shutil.copy(video_path, standard_video_path)
    print(f"Copied uploaded video to standard name: {standard_video_

# ============================================================
# 5. FINAL STATUS
# ============================================================
print("\nSetup done!")
print(f"Repo path: {REPO}")
print(f"Selected video: {video_path}")
print(f"Standard video path for downstream code: {standard_video_pat
```

```
Repo already exists
Dependencies ready
Default video not found. Please upload an MP4 file.
```

Choose Files output_ds_2.mp4

output_ds_2.mp4(video/mp4) - 56187929 bytes, last modified: 4/11/2026 - 100% done

```
Saving output_ds_2.mp4 to output_ds_2.mp4
Using uploaded video: /content/AAI-590-Capstone-group2/output_ds_2.mp4
Video size: 56.2 MB
Copied uploaded video to standard name: /content/AAI-590-Capstone-group2/

Setup done!
Repo path: /content/AAI-590-Capstone-group2
Selected video: /content/AAI-590-Capstone-group2/output_ds_2.mp4
Standard video path for downstream code: /content/AAI-590-Capstone-group2
```

In this code 5 steps are applied in sequence:

- Step 1 — Counter variables: Adds tracking variables before the main loop — counters for LISA detections, COCO detections, left/right blind spot alerts, fast-approach events, frames with detections, and a list to record per-frame render times.
- Step 2 — Frame timer start: Records a timestamp _ft at the top of every frame loop iteration to measure how long each frame takes to render.
- Step 3 — Detection counters: After each detection is processed, increments the appropriate counter — by model source (LISA vs COCO), by blind spot side (left/right), and by fast-approach flag.
- Step 4 — Frame timing capture: After each frame is written to the output video, appends the elapsed render time in milliseconds and counts whether the frame had any detections.
- Step 5 — Save performance JSON: After the pipeline finishes, computes summary statistics (avg FPS, avg render time, detection rate, alert counts, tracked objects)

and saves everything to /content/perf_{mode}.json for later analysis.

- Finally, a verification pass checks all 5 steps landed correctly and prints a clear pass/fail for each before allowing the pipeline to run.

Cell #2

```

import os
REPO = "/content/AAI-590-Capstone-group2"

# Step 1: restore original from git first (removes any previous pat
os.system(f"cd {REPO} && git checkout omniview_e2e_live.py")
print("Original file restored")

code = open(f"{REPO}/omniview_e2e_live.py").read()

# ——1. Add performance counter variables ——————————————
old1 = "    vel_tracker = VelocityTracker()\n\n    total = 0\n    t
new1 = """    vel_tracker = VelocityTracker()

    total            = 0
    lisa_total       = 0
    coco_total       = 0
    left_alert_total = 0
    right_alert_total= 0
    fast_app_total   = 0
    frames_with_dets = 0
    frame_times      = []
    t0               = time.time()
    i                = 0"""

# 2: start frame timer at top of loop ————————————————
old2 = "        for i in range(max_frames):\n            if _canvas
new2 = "        for i in range(max_frames):\n            _ft = time

# 3: count detections by model/zone ——————————————————
old3 = "                total += 1\n\n            if instruction:"
new3 = """                total += 1
                if model_src == "lisa": lisa_total += 1
                else:                   coco_total += 1
                if bs == "left":        left_alert_total  += 1
                if bs == "right":       right_alert_total += 1
                if fast_approach:       fast_app_total    += 1

            if instruction:"""

# 4: record frame render time after writer.write ————
```

```
old4 = "            writer.write(frame)\n\n            if i % 100 =
new4 = """            writer.write(frame)

            frame_times.append((time.time() - _ft) * 1000)
            if len(dets) > 0:
                frames_with_dets += 1

            if i % 100 == 0:"""

# 5: save perf data to JSON file ———————————————————
old5 = '        print(f"[Pipeline] MP4 saved: {out_path}"\n
new5 = '''        try:
            mp4_kb = os.path.getsize(str(out_path)) // 1024
            print(f"[Pipeline] MP4 saved: {out_path} ({mp4_kb:,} KB
        except Exception:
            mp4_kb = 0
        try:
            _ft_arr = np.array(frame_times) if frame_times else np.
            _perf = {
                "mode":             args.mode.upper(),
                "resolution":       f"{w} x {h} @ {fps:.0f} fps",
                "total_frames":     i + 1,
                "elapsed_s":        round(elapsed, 2),
                "avg_fps":          round((i+1)/elapsed, 2),
                "avg_render_ms":    round(float(np.mean(_ft_arr)),
                "min_render_ms":    round(float(np.min(_ft_arr)), 2
                "max_render_ms":    round(float(np.max(_ft_arr)), 2
                "total_dets":       total,
                "avg_dets_frame":   round(total / max(i+1,1), 2),
                "frames_with_dets": frames_with_dets,
                "det_rate_pct":     round(frames_with_dets / max(i+
                "lisa_total":       lisa_total,
                "coco_total":       coco_total,
                "left_alerts":      left_alert_total,
                "right_alerts":     right_alert_total,
                "fast_approach":    fast_app_total,
                "tracked_objects":  len(vel_tracker.history),
                "mp4_kb":           mp4_kb,
            }
            import json as _json
            _perf_path = f"/content/perf_{args.mode}.json"
            with open(_perf_path, "w") as _pf:
                _json.dump(_perf, _pf)
            print(f"[Pipeline] Perf saved -> {_perf_path}")
        except Exception as _e:
            print(f"[Pipeline] Perf save failed: {_e}")'''

# —— Apply all steps from above3 ————————————————————————————
steps = [(old1,new1),(old2,new2),(old3,new3),(old4,new4),(old5,new5
for idx, (old, new) in enumerate(steps, 1):
    if old in code:
```

```
        code = code.replace(old, new)
        print(f" Step {idx} applied")
    else:
        print(f" Step {idx} NOT FOUND — re-clone repo and retry")

with open(f"{REPO}/omniview_e2e_live.py", "w") as f:
    f.write(code)

# ── Verify ──────────────────────────────────────────────────
print()
checks = [
    ("Counter vars",       "frame_times      = []"),
    ("Frame timer",        "_ft = time.time()"),
    ("Detection counters", "lisa_total += 1"),
    ("Frame timing",       "frame_times.append"),
    ("Perf save",          "perf_{args.mode}"),
]
all_ok = True
for name, check in checks:
    ok = check in code
    print(f"{'Ok' if ok else 'X'}  {name}")
    if not ok:
        all_ok = False

print()
print(" All Steps applied — ready to run!" if all_ok else "X Some S
```

```
Original file restored
 Step 1 applied
 Step 2 applied
 Step 3 applied
 Step 4 applied
 Step 5 applied

Ok  Counter vars
Ok  Frame timer
Ok  Detection counters
Ok  Frame timing
Ok  Perf save

 All Steps applied — ready to run!
```

OmniView — Real-Time AR Driver Awareness System

Edge-deployed augmented reality driver assistance system that detects vehicles, pedestrians, and traffic signs in real time and overlays safety alerts onto the camera feed — no cloud required.

Pipeline: - Dual-model DeepStream pipeline on NVIDIA Jetson Orin Nano - COCO model: vehicle and pedestrian detection - LISA model: traffic sign recognition - Both models run simultaneously on the same edge device

HUD Layers (rendered via OpenCV on every frame):

- 1. Status bar — pipeline mode and frame sequence number
- 2. Blind spot zones — cyan overlays, lower 52% height, 22% width each side
- 3. Bounding boxes — color-coded by threat level
- 4. Traffic sign — orange banner when LISA detects a sign
- 5. Blind spot alert — red banner when a vehicle enters a zone

Box Colors: Green — safe, high confidence Yellow — caution, low confidence (<50%) Red — critical blind spot Orange — fast-approaching vehicle

VelocityTracker: Tracks each object across frames using a grid-based key (label + position). Computes dx, dy, speed, and bounding box area change per frame. Assigns one of six motion labels: STATIC — barely moving APPROACH — box growing (closing in) RECEDING — box shrinking (moving away) MERGING-LEFT — lateral movement left MERGING-RIGHT— lateral movement right MOVING — in motion, direction unclear Fast-approach flag fires when area growth > 15% and speed > 3 px/frame.

Pipeline Modes: Mode 1 (File) — reads pre-recorded JSON detections for offline testing Mode 2 (UDP) — receives live detections from DeepStream over UDP port 5055

Performance (validated at 1920x1080): File mode : 7.36 fps | 6,982 detections | 99.9% accuracy | 2,117 blind spot alerts UDP mode : 4.38 fps | +40 LISA signs | | 1,968 blind spot alerts

Mode 1 cell (Json file mode)

```
import os
os.chdir("/content")
REPO = "/content/AAI-590-Capstone-group2"

result = __import__('subprocess').run([
    "python3", f"{REPO}/omniview_e2e_live.py",
    "--no-bootstrap", "--mode", "file",
    "--json",   f"{REPO}/runs/hud/ds_1",
    "--video",  f"{REPO}/output_ds_1.mp4",
    "--output", "/content/omniview_live_hud.mp4",
    "--gif-frames", "40", "--gif-fps", "10",
```

```
], capture_output=True, text=True)
print(result.stdout[-2000:])
```

```
============================================================
  OmniView AI -- End-to-End Live HUD  v4.0
  AAI-590 Capstone Group 2 | USD
============================================================
  Victor's omniview_pipeline.py loaded
     get_blind_spot_zones, draw_blind_spot_zones
     get_box_color, draw_alerts
     DetectionPayload, build_message
  Sunitha's VelocityTracker + INSTRUCTION_MAP loaded
  Tej's ds_pipeline.py referenced for Mode 3 (Jetson)
     Cannot import on Colab -- Jetson hardware required

  Mode   : FILE
  Video  : /content/AAI-590-Capstone-group2/output_ds_1.mp4
  JSON   : /content/AAI-590-Capstone-group2/runs/hud/ds_1
  Output : /content/omniview_live_hud.mp4
============================================================
[Pipeline] Video  : /content/AAI-590-Capstone-group2/output_ds_1.mp4  192
[Pipeline] JSON   : 1107 frames from /content/AAI-590-Capstone-group2/run
[Pipeline] Total  : 1107 frames
frame=0/1107  dets=6  total=6  0.0fps
frame=100/1107  dets=3  total=705  5.9fps
frame=200/1107  dets=5  total=1240  6.7fps
frame=300/1107  dets=6  total=1803  6.5fps
frame=400/1107  dets=6  total=2369  6.6fps
frame=500/1107  dets=4  total=3054  6.7fps
frame=600/1107  dets=6  total=3688  6.8fps
frame=700/1107  dets=7  total=4403  6.9fps
frame=800/1107  dets=6  total=4998  6.9fps
frame=900/1107  dets=6  total=5601  7.0fps
frame=1000/1107  dets=5  total=6307  7.0fps

[Pipeline] Done. 6913 detections in 1097 frames (7.1fps)
[Pipeline] MP4 saved: /content/omniview_live_hud.mp4 (237,771 KB)
[Pipeline] Perf saved -> /content/perf_file.json

[Pipeline] Creating GIF...
[Pipeline] GIF saved: /content/omniview_live_hud.gif (4.0s)
<IPython.core.display.Image object>
```

Deep Stream

Before running this cell, run set up cell and cell #2 THis allows erase parameters and variables belong to Mode 1 first to prepare for Mode 2 (UDP Mode)

- This script launches the DeepStream detection pipeline (omniview_pipeline.py), which is the AI backbone that processes video frames through the dual YOLO

models and broadcasts detections over UDP port 5055. The previous script launches the HUD renderer (omniview_e2e_live.py in UDP mode), which listens on that same port to receive those detections and draw the overlay.

- This script first — starts DeepStream, patches paths, disables display/recording, and waits up to 90 seconds until it confirms the pipeline is running Previous script second — starts the HUD renderer, connects to UDP 5055, and processes the incoming detections

Deep Stream Cell

```
import subprocess, threading, time
REPO  = "/content/AAI-590-Capstone-group2"
VIDEO = f"{REPO}/output_ds_1.mp4"

with open(f"{REPO}/victor_deepstream/omniview_pipeline.py") as f:
    code = f.read()
code = code.replace("DISPLAY      = True",  "DISPLAY      = False")
code = code.replace("RECORD       = True",  "RECORD       = False")
code = code.replace("/home/logicpro09/omniview_ai/yolov8n_lisa_v1.1.
                    f"{REPO}/victor_deepstream/yolov8n_lisa_v1.1.onn
code = code.replace("/home/logicpro09/omniview_ai/yolov8n.pt",
                    f"{REPO}/models/yolov8n.pt")
code = code.replace("/home/logicpro09/omniview_ai/output_ds_3_reenc.
code = code.replace("output_ds_3_reenc.mp4", VIDEO)
with open("/content/omniview_pipeline_patched.py", "w") as f:
    f.write(code)

proc = subprocess.Popen(
    ["python3", "-u", "/content/omniview_pipeline_patched.py"],
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
    text=True, bufsize=1
)
victor_started = threading.Event()
def _stream(p):
    for line in p.stdout:
        print(f"[Victor] {line}", end="", flush=True)
        if "fps" in line.lower() or "frame" in line.lower():
            victor_started.set()
threading.Thread(target=_stream, args=(proc,), daemon=True).start()
print("Waiting for Victor deepstream(up to 90s)...")
victor_started.wait(timeout=90)
print("Victor deep stream running! Run next cell now.")
```

```
Waiting for Victor deepstream(up to 90s)...
[Victor] WARNING ⚠️ Unable to automatically guess model task, assuming 't
[Victor] JSON files -> /home/logicpro09/omniview_ai/runs/hud/ds_3
```

```
[Victor] Starting OmniView AI pipeline - dual model...
[Victor] LISA model: traffic sign detection
[Victor] COCO model: vehicle/pedestrian blind spot detection
[Victor] UDP stream -> 127.0.0.1:5055
[Victor] Video: /content/AAI-590-Capstone-group2/output_ds_1.mp4
[Victor] ----------------------------------------------------
[Victor] Loading /content/AAI-590-Capstone-group2/victor_deepstream/yolov
[Victor] requirements: Ultralytics requirements ['onnx', 'onnxruntime'] n
[Victor] Using Python 3.12.13 environment at: /usr
[Victor] Resolved 10 packages in 432ms
[Victor] Prepared 2 packages in 3.00s
[Victor] Installed 2 packages in 470ms
[Victor]  + onnx==1.21.0
[Victor]  + onnxruntime==1.24.4
[Victor]
[Victor] requirements: AutoUpdate success ✅ 4.8s
[Victor] WARNING ⚠️ requirements: Restart runtime or rerun command for up
[Victor]
[Victor] Using ONNX Runtime 1.24.4 with CPUExecutionProvider
Victor deep stream running! Run next cell now.
```

This script launches the OmniView HUD pipeline in UDP Live mode (Mode 2) as a subprocess. Runs omniview_e2e_live.py with the following configuration:

- --no-bootstrap — skips the DeepStream startup sequence, assuming the pipeline is already running
- --mode udp — sets Mode 2, receiving live detections over UDP instead of reading from JSON files
- --video — provides the source video file for HUD rendering
- --output — saves the annotated result to /content/omniview_live_hud_udp.mp4
- --gif-frames 40 --gif-fps 10 — generates a preview GIF from the first 40 frames at 10 fps
- --udp-host / --udp-port — listens on 127.0.0.1:5055 for incoming DeepStream detection packets
- --udp-wait 30 — waits up to 30 seconds for the first UDP packet before timing out
- --max-frames 2203 — caps processing at 2,203 frames

After execution, proc.terminate() stops any lingering process and the last 2,000 characters of stdout are printed for a quick pipeline summary check.

The following cell will run for atleast 5 to 10 mins

Mode 2 Cell (UDP Mode)

```
import subprocess
REPO = "/content/AAI-590-Capstone-group2"

result = subprocess.run([
    "python3", f"{REPO}/omniview_e2e_live.py",
    "--no-bootstrap", "--mode", "udp",
    "--video",      f"{REPO}/output_ds_1.mp4",
    "--output",     "/content/omniview_live_hud_udp.mp4",
    "--gif-frames", "40", "--gif-fps", "10",
    "--udp-host",   "127.0.0.1",
    "--udp-port",   "5055",
    "--udp-wait",   "30",
    "--max-frames", "2203",
], capture_output=True, text=True)
proc.terminate()
print(result.stdout[-2000:])
```

```
============================================================
  OmniView AI -- End-to-End Live HUD  v4.0
  AAI-590 Capstone Group 2 | USD
============================================================
  Victor's omniview_pipeline.py loaded
     get_blind_spot_zones, draw_blind_spot_zones
     get_box_color, draw_alerts
     DetectionPayload, build_message
  Sunitha's VelocityTracker + INSTRUCTION_MAP loaded
  Tej's ds_pipeline.py referenced for Mode 3 (Jetson)
     Cannot import on Colab -- Jetson hardware required

  Mode   : UDP
  Video  : /content/AAI-590-Capstone-group2/output_ds_1.mp4
  JSON   : /content/AAI-590-Capstone-group2/runs/hud/ds_1
  Output : /content/omniview_live_hud_udp.mp4
============================================================
[Pipeline] Video  : /content/AAI-590-Capstone-group2/output_ds_1.mp4  192
[UDP] Listening on 127.0.0.1:5055
[Pipeline] UDP mode -- waiting 30.0s for first packet...
frame=0/2203  dets=1  total=1  0.0fps
frame=100/2203  dets=0  total=16  3.6fps
frame=200/2203  dets=3  total=186  3.9fps
frame=300/2203  dets=6  total=641  4.1fps
frame=400/2203  dets=6  total=1268  4.1fps
frame=500/2203  dets=3  total=1784  4.2fps
frame=600/2203  dets=4  total=2197  4.2fps
frame=700/2203  dets=2  total=2380  4.2fps
frame=800/2203  dets=3  total=2605  4.2fps
frame=900/2203  dets=4  total=2909  4.3fps
frame=1000/2203  dets=3  total=3265  4.3fps

[Pipeline] Done. 3632 detections in 1097 frames (4.3fps)
[Pipeline] MP4 saved: /content/omniview_live_hud_udp.mp4 (217,963 KB)
[Pipeline] Perf saved -> /content/perf_udp.json

[Pipeline] Creating GIF...
[Pipeline] GIF saved: /content/omniview_live_hud_udp.gif (4.0s)
```

```
<IPython.core.display.Image object>
```

Report Cell

Report generation

```
import json, os
from IPython.display import display, HTML

perf_file = "/content/perf_file.json"
perf_udp  = "/content/perf_udp.json"

if os.path.exists(perf_udp) and os.path.exists(perf_file):
    p_path = perf_udp if os.path.getmtime(perf_udp) > os.path.getmti
elif os.path.exists(perf_udp):
    p_path = perf_udp
else:
    p_path = perf_file

with open(p_path) as f:
    p = json.load(f)

print(f"Showing: {p['mode']} mode")

rows = [
    ("section", "Pipeline & Render"),
    ("Mode",                        p["mode"]),
    ("Video resolution",            p["resolution"]),
    ("Total frames processed",      f"{p['total_frames']:,}"),
    ("Total runtime",               f"{p['elapsed_s']} s"),
    ("Average render FPS",          f"{p['avg_fps']} fps"),
    ("Avg frame render time",       f"{p['avg_render_ms']} ms"),
    ("Min frame render time",       f"{p['min_render_ms']} ms"),
    ("Max frame render time",       f"{p['max_render_ms']} ms"),
    ("Output MP4 size",             f"{p['mp4_kb']:,} KB"),
    ("section", "Detection Metrics"),
    ("Total detections drawn",      f"{p['total_dets']:,}"),
    ("Avg detections / frame",      f"{p['avg_dets_frame']}"),
    ("Frames with detections",      f"{p['frames_with_dets']:,}"),
    ("Detection rate",              f"{p['det_rate_pct']} %"),
    ("LISA detections (signs)",     f"{p['lisa_total']:,}"),
    ("COCO detections (vehicles)",  f"{p['coco_total']:,}"),
    ("section", "HUD & Alert Metrics"),
    ("Left blind spot alerts",      f"{p['left_alerts']:,}"),
    ("Right blind spot alerts",     f"{p['right_alerts']:,}"),
    ("Total blind spot alerts",     f"{p['left_alerts']+p['right_ale
```

```
    ("Fast-approach events",        f"{p['fast_approach']:,}"),
    ("Unique objects tracked",      f"{p['tracked_objects']:,}"),
]

html = """<div style='background:#0A2A35;padding:24px;border-radius:
            border:2px solid #41AEBD;font-family:monospace;max-width
  <h2 style='color:#41AEBD;text-align:center;letter-spacing:3px;marg
      OmniView AI — HUD Performance Report</h2>
  <p style='text-align:center;color:#7FB8C8;font-size:12px;margin-to
      Mode: {mode}  |  {res}  | 
      {frames} frames  |  {rt}s runtime</p>
  <table style='border-collapse:collapse;width:100%;font-size:13px'>
    <tr style='background:#41AEBD;color:#0A2A35'>
      <th style='padding:8px 14px;text-align:left'>Metric</th>
      <th style='padding:8px 14px;text-align:right'>Value</th></tr>"
    mode=p["mode"], res=p["resolution"],
    frames=f"{p['total_frames']:,}", rt=p["elapsed_s"])

section_colors = {
    "Pipeline & Render":   "#41AEBD",
    "Detection Metrics":   "#4CAF82",
    "HUD & Alert Metrics": "#F5A623",
}
row_idx = 0
for item in rows:
    if item[0] == "section":
        col = section_colors.get(item[1], "#41AEBD")
        html += f"<tr style='background:{col}22'><td colspan='2' sty
        row_idx = 0
    else:
        k, v = item
        bg = "#0D3D50" if row_idx % 2 == 0 else "#0A2A35"
        html += f"<tr style='background:{bg}'><td style='padding:6px
        row_idx += 1

html += "</table></div>"
display(HTML(html))
```

```
Showing: UDP mode
```

OmniView AI — HUD Performance Report

Mode: UDP | 1920 x 1080 @ 30 fps | 1,097 frames | 255.83s runtime

Metric	Value
▸ **Pipeline & Render**	
Mode	UDP
Video resolution	1920 x 1080 @ 30 fps
Total frames processed	1,097
Total runtime	255.83 s
Average render FPS	4.29 fps
Avg frame render time	233.41 ms
Min frame render time	169.59 ms
Max frame render time	5595.49 ms
Output MP4 size	217,963 KB
▸ **Detection Metrics**	
Total detections drawn	3,632
Avg detections / frame	3.31
Frames with detections	981
Detection rate	89.4 %
LISA detections (signs)	41
COCO detections (vehicles)	3,591
▸ **HUD & Alert Metrics**	
Left blind spot alerts	470
Right blind spot alerts	315
Total blind spot alerts	785
Fast-approach events	475
Unique objects tracked	33

GIF Run cell

This script displays the output GIFs from both pipeline modes side by side in the Colab notebook. It loops through two expected GIF files — one from Mode 1 (File) and one from Mode 2 (UDP). For each, it checks if the file exists, prints the label and file size in MB, then renders it inline in the notebook using IPython's Image display. If a file is missing, it prints a not-found message instead.

- This serves as a quick visual sanity check to confirm both pipeline modes produced

```
from IPython.display import Image, display, HTML
import os

gifs = {
    "Mode 1 — File": "/content/omniview_live_hud.gif",
    "Mode 2 — UDP":  "/content/omniview_live_hud_udp.gif",
}

for label, path in gifs.items():
    if os.path.exists(path):
        size = os.path.getsize(path) / 1e6
        print(f"{label}  ({size:.1f} MB)")
        display(Image(filename=path))
    else:
```